

Erste Schritte mit Databricks Apps

Ein Schritt-für-Schritt-Tutorial, um die erste eigene App zu erstellen, lokal zu bearbeiten und bereitzustellen.

Diese Seite zeigt anhand eines konkreten Beispiels ("Gradio Hello World"-Vorlage), wie man in Databricks eine App erstellt, sie lokal auf dem eigenen Rechner bearbeitet und anschließend wieder in den Workspace zurückspielt. Das Beispiel eignet sich als praktischer Einstieg, bevor man eigene Apps von Grund auf entwickelt.

Voraussetzungen

Bevor es losgeht, muss der Databricks-Workspace und die lokale Entwicklungsumgebung korrekt eingerichtet sein (siehe die Seite "Entwicklungsumgebung einrichten").

Schritt 1: Abhängigkeiten installieren

Im Terminal werden zunächst die benötigten Python-Pakete installiert und ein lokaler Projektordner angelegt:

```
pip install gradio
pip install pandas
mkdir gradio-hello-world
cd gradio-hello-world
```

Der Ordner `gradio-hello-world` dient als lokaler Speicherort für alle App-Dateien.

Schritt 2: App erstellen

In der Databricks-Oberfläche wechselt man über den App-Switcher zu "Databricks Apps", klickt auf "Erstellen", wählt die Vorlage "Gradio Hello World" aus und vergibt den Namen `gradio-hello-world`. Bei GovCloud-Workspaces findet man den Zugang über die Seitenleiste "Compute" im Reiter "Apps".

Nach der Erstellung wird die App automatisch auf Basis der gewählten Vorlage bereitgestellt (deployed).

Schritt 3: App ansehen

Nach dem Deployment zeigt die Übersichtsseite die automatisch generierte App-URL sowie den Bereitstellungsstatus. Databricks erzeugt die URL automatisch aus App-Name und Workspace-ID. Diese URL kann nachträglich **nicht geändert** werden – wer eine andere URL braucht, muss eine neue App mit anderem Namen anlegen.

Schritt 4: App auf den eigenen Rechner kopieren

Über den Bereich "Edit in your IDE" auf der App-Detailseite erhält man die passenden Export-Befehle. Führt man diese im lokalen Ordner `gradio-hello-world` aus, werden drei zentrale Dateien heruntergeladen:

```
databricks workspace export-dir /Workspace/Users/[pfad]/gradio-hello-world-app
databricks sync --watch . /Workspace/Users/[pfad]/gradio-hello-world-app
```

Datei	Zweck
<code>app.py</code>	Enthält die Programmlogik und die Benutzeroberfläche
<code>app.yaml</code>	Definiert Konfiguration, Einstiegspunkt (Startbefehl) und Berechtigungen
<code>requirements.txt</code>	Liste der benötigten Python-Pakete

Der `sync`-Befehl überwacht laufend lokale Änderungen und lädt sie automatisch in den Workspace hoch.

Schritt 5: Lokal bearbeiten und testen

Man kann die App nun lokal anpassen und testen:

```
python app.py
```

Das startet eine lokale Instanz unter `http://127.0.0.1:7860`.

Alternativ steht ein spezieller Testbefehl zur Verfügung, der die Umgebung automatisch vorbereitet:

```
databricks apps run-local --prepare-environment --debug
```

Dieser installiert Abhängigkeiten, bereitet eine virtuelle Umgebung vor und startet die App mit aktiviertem Debugging auf Port 5678, erreichbar unter `http://localhost:8001`. In Visual Studio Code lassen sich mit der Python-Debugger-Erweiterung über "Run > Start Debugging > Remote Attach" Haltepunkte (Breakpoints) setzen.

Schritt 6: Erneut im Workspace bereitstellen

Den Deployment-Befehl kopiert man von der Übersichtsseite der App und führt ihn im lokalen Ordner aus. Bei Erfolg liefert die Konsole eine JSON-Bestätigung mit Deployment-ID, Status und Zeitstempeln zurück.

Schritt 7 (optional): Bereitstellung über ein Git-Repository

Für professionellere Arbeitsabläufe mit Versionskontrolle und Zusammenarbeit im Team lohnt sich die Anbindung an Git:

```
git init
git add .
git commit -m "Initial app code"
git remote add origin https://github.com/your-org/gradio-hello-world
git push -u origin main
```

Danach verknüpft man das Git-Repository mit der App:

```
databricks apps create-update gradio-hello-world \
  --json '{"update_mask": "git_repository", "git_repository": {"url": "https://github.com/your-org/gradio-hello-world.git", "branch": "main"}}'
```

Bei privaten Repositories müssen zusätzlich Git-Zugangsdaten für das Service Principal (die technische Identität der App) hinterlegt werden. Das Deployment erfolgt dann mit:

```
databricks apps deploy gradio-hello-world \
  --json '{"git_source": {"branch": "main"}}'
```

Tipp: Wer direkt mit Git-Deployment arbeiten möchte, kann die Vorlage überspringen und stattdessen gleich eine individuelle App erstellen.

Wichtige Hinweise

- Die Synchronisierung mit dem Workspace lädt lokale Änderungen während der Entwicklung automatisch hoch.
- Optional lässt sich ein automatisches Deployment bei jedem Git-Push einrichten.
- Apps lassen sich mit weiteren Databricks-Funktionen verbinden, etwa Secrets (verschlüsselte Zugangsdaten), SQL-Datenbanken und feingranularer Autorisierung.
- Für erste Experimente stehen weitere Vorlagen mit unterschiedlichen Frameworks zur Verfügung.

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/get-started>
Stand: 2026-08-06